PA195DC

A materially different 195-state binary Turing machine for the consistency of Peano arithmetic

Research prepared for Vladimir Reshetnikov

13 November 2026

ProveIt snapshot `5ed5a41fa5932433111eea096311364c6087e8e3`

Abstract

We compare the current PA-machine corpus and construct a new deterministic, single-tape, two-symbol machine with 195 working states. We did not obtain a verified machine with fewer than 195 states. The delivered machine fulfills the alternative target: it combines the original PA195 diagonal axiom emitter and its seven contextual reader folds with the materially different Combined proof-search controller. It changes six literal transition rows, removes 24 primitive source instructions, and changes the complete blank startup. It is not isomorphic to any of the 422 spanning action-graph classes in the pinned corpus, even after tape reflection or simultaneous bit complementation.

The construction has 390 transition rows and a 3,676-instruction source program. Its deterministic checker reproduces the supplied PA195 parent, the exact changes, all 4,784 symbolic source outcomes, all 28 two-cell fold cases, and the complete 57,922-transition startup. An independent physical checker rederives the register-list lemmas from the literal table. The report supplies the mathematical argument that PA proves nonhalting of the new table equivalent to Con(PA), with explicit attribution for the inherited logical calibration and controller compiler. A newly kernel-checked Lean or Rocq proof of the whole biconditional is not claimed.

195 working states 2 symbols 390 transitions

$$\mathrm{PA} \vdash \neg\,\mathrm{Halt}(\mathrm{PA195DC}, \mathrm{blank}) \leftrightarrow \mathrm{Con}(\mathrm{PA}).$$

A new same-size construction; no claim of a 194-state result.

Contents

1 Result, exact conventions, and proof status **3**

2 Comparison of the complete pinned corpus **4**

3 The new literal construction **5**

4 The unchanged numerical calculus **6**

5 Arithmetic calibration of the unchanged diagonal calculus **7**

6 Why the changed controller still enumerates refutations **11**

7 Binding the proof to the literal binary machine **14**

8 The search below 195 and what the negative results mean **17**

9 Reproduction, provenance, and limits **18**

A Complete transition table **20**

B Archive-by-archive comparison **21**

C Safe changes to the axiom emitter **25**

D Why replacing the equality witness by reflexivity is not immediate **26**

1 Result, exact conventions, and proof status

The definitive object is `machine/machine.tm`. The working controls are $0, \ldots, 194$, control 0 is initial, and the separate halt label is 195. The alphabet is $\{0, 1\}$, the tape is indexed by $\mathbb{Z}$, the initial head position is zero, and every initial cell is zero. A row (q, b, w, D, q') means read b, write w, move one cell in direction $D \in \{L, R\}$, and enter q'. There are no stationary moves and no outgoing transitions from halt. The halt label is excluded from the count.

Every working control is reachable in the finite transition graph from the start. This graph statement does not assert that every control occurs in the actual blank computation. The complete table is printed in Appendix A. Its native-label presentation has start 74 and halt 254; `machine/labels.json` is an exact dense-to-native conjugacy.

Quantity	Value
Working states / binary transitions	195 / 390
Unfolded working controls	202
Fixed controls / AX readers / folds	134 / 68 / 7
Source instructions / branch contracts	3,676 / 4,784
AX macro tokens	191
Largest source-address length	38
Exact blank startup	57,922 transitions

The SHA-256 of the dense five-column table, including its final newline, is

`e5cc1b2cb374fa4342070e95d8bdfd60c1bb845e7a52d44857bdee937bba7938`.

Theorem 1.1. *Let M_{DC} be that exact table and let H_{DC} be the arithmetic sentence asserting a finite halting trace from its specified blank start. For an ordinary fixed primitive-recursive PA proof checker,*

$$\mathrm{PA} \vdash H_{\mathrm{DC}} \leftrightarrow \exists p \, \mathrm{Prf}_{\mathrm{PA}}(p, \ulcorner 0 = 1 \urcorner).$$

Consequently $\mathrm{PA} \vdash \neg H_{\mathrm{DC}} \leftrightarrow \mathrm{Con(PA)}$.

The claim is a mathematical proof of internal PA provability. It is stronger than an external statement that the machine does not halt in the standard model. Its proof uses syntactic proof transformations and induction on coded finite computations. In particular, it does not invoke PA's reflection principle or assume Con(PA) to prove the biconditional.

Three levels of evidence are distinguished throughout. The repository contains substantial inherited Lean proofs, including the PA196 implication from nonhalting to consistency and the original PA195 native-table quotient. The complete new biconditional is proved in ordinary mathematical form here by the stated interfaces. The delivered Python programs are executable finite and symbolic certificates binding that argument to the literal table. They are not represented as a newly completed proof-assistant theorem of the whole endpoint. This distinction is a formalization limit, not a claim that finite simulation alone establishes consistency equivalence.

2 Comparison of the complete pinned corpus

The comparison uses ProveIt commit `5ed5a41fa5932433111eea096311364c6087e8e3`, not a moving branch. All 32 top-level ZIP archives in `docs/incoming` were inventoried, including recursively nested archives, together with the PA research tree [1]. Appendix B discusses every current archive. The machine-readable comparison preserves archive hashes, member paths, graph classes, state counts, and the distinction between proposed outputs, ancestors, failed experiments, and synthetic controls.

The census finds 45 distinct ZIP payloads through 73 archive references, with maximum nesting depth eight. It reads 2,363 occurrences of strict numeric binary transition tables, including 14 repository files. There are 583 distinct byte strings and 573 distinct labeled transition functions. Root-free canonicalization gives 422 action-graph classes with a vertex reaching every working state, plus two disconnected synthetic controls of sizes six and nine. A four-state positive control is included among the 422. These counts are not counts of successful consistency machines.

The literal census recognizes `.tm` files and strict five-column numeric tables in `.txt`, `.tsv`, and `.csv`; it does not pretend to enumerate every table a Python program might generate or every table encoded only as a Lean or JSON expression. The manuscripts and formal proof directories were reviewed separately for their substantive designs.

2.1 The actual 195-state frontier

There are 37 literal 195-state occurrences, 14 labeled functions, and nine action-graph classes. Three classes are supported constructions in the reports; the other six are explicitly rejected diagnostics. The three supported designs must all be included in the baseline:

Design	Sites	AX word	Readers	Folds	States	Startup
PA195, diagonal	3,700	191	68	7	195	58,154
195S, separate head	3,673	190	67	6	195	58,154
195C, separate + Combined	3,649	190	67	6	195	57,922
PA195DC, this report	**3,676**	191	68	7	**195**	57,922

Original PA195 saves a seventh reader state in a 202-control unfolding. 195S uses a shorter, separate-diagonal-head emitter and a 201-control unfolding with six folds. 195C combines that emitter with a changed Shift and MP controller. The new PA195DC uses the original *bundled* diagonal emitter with the changed controller. Its source is 24 instructions shorter than original PA195, but it is not the shortest source among the four designs; source length and physical state count are different metrics.

The new graph is absent from all 422 spanning classes under canonical comparison that even ignores the designated start control. Neither tape reflection, simultaneous exchange of read/write bits, nor their combination makes it equal to a supplied class. The retained canonical hash and complete comparison receipt are in `comparison/`. The change is more than a relabeling: the register-command transition relation itself changes.

2.2 Larger machines and nonadditive reductions

James Chang's PA209 predecessor and the U209 ancestor of the U208 reports are different source programs. The U208 reduction merges native roles 172 and 211 using entry history and an existing helper. Repeated reports of this fold do not yield repeated savings; later helper choices give 20 designated U208 action graphs. Those roles are already absent from the PA196 lineage, so the old fold cannot simply be subtracted again.

The PA196 frontier includes alternate head orders, binary associations, physical register exchanges, a forced-zero jump, several closing decrement variants, H-only Shift, MP-result-in-H, and their combinations with the diagonal and separate-head calculi. Smaller source DAGs sometimes lose a helper or need a longer scanner. A seven-zero source address can be mistaken for a marker; repairing the scanner can restore the original state count. The closing CD3–CD6 proposals all change the same transition and therefore are alternatives. A zero-jump at a different row can be composed only after checking the resulting full machine.

Among the failed 195-state classes are a two-rotation quotient with 93 failed source contracts, two physical folds with 714 and 777 failures, a seven-zero scanner error, a separate-head scanner with 96 failed unrestricted contracts, and an incorrect zero-jump opcode. These refute their advertised simulations; they are not universal impossibility proofs about every interpretation of those tables. Two other rejected Shift layouts give stronger evidence: they halt on the blank tape after 43,563 steps.

The eight `pa195-*` proof directories develop successive formal layers for the original PA195 quotient, not eight distinct designs. The native-table any-tape halting theorem in `pa195-global` explicitly stops short of dense conjugacy and the arithmetic consistency endpoint [8].

3 The new literal construction

3.1 Six row changes and a shorter command tail

Start from the supplied diagonal PA195 [3]. Keep its complete AX grammar, its 191-token word, all parameter assignments and emitted formulas, and the seven contextual reader folds. In native coordinates change exactly the following six transitions:

State	Read	Old action	New action
38	0	$0R28$	$0R11$
38	1	$1R20$	$1R122$
164	1	$1R98$	$1R130$
214	0	$0R188$	$0R312$
214	1	$1R312$	$1R20$
312	0	$0R11$	$0R188$

All other native rows are unchanged. The two source DAG substitutions are

$$\texttt{node163} = (\texttt{node6}, \texttt{node147}),$$

$$\texttt{node170} = ((\texttt{node36}, (\texttt{node167}, \texttt{node145})), \texttt{node57}).$$

These are the inherited Combined controller edits, now checked in combination with the seven-fold diagonal allocation. Their exact binding to ordinary register instructions is supplied, rather than inferred from the names.

Both source-graph resolvers agree on all 3,676 instructions. The first 3,661 instructions preserve the old opcode, register, and ordinal successors except for three retargeted exits: the zero branch at 48 goes to 3665, and the jumps at 3580 and 3660 go to 3664. The final 15 instructions implement the new MP success continuation and Shift. No AX emission instruction changes.

3.2 Why the count remains 195

The unfolded allocation has 134 fixed controls and 68 AX readers. The seven disjoint identifications, with the deleted control on the left, are

$$2018 \mapsto 2024, \quad 2030 \mapsto 2050, \quad 2038 \mapsto 2019,$$
$$2042 \mapsto 2028, \quad 2053 \mapsto 2043, \quad 2055 \mapsto 2034, \quad 2062 \mapsto 2036.$$

Thus $134 + 68 - 7 = 195$ working controls remain. The six controller changes do not modify any arithmetic-box row or any of the fold-state entry premises. Nevertheless the complete folded table is rechecked; compatibility is not assumed from the fact that each separate optimization worked elsewhere.

Lemma 3.1 (Global seven-fold simulation)**.** *The delivered unfolded 202-state native table and PA195DC halt on exactly the same complete binary tapes when started in native control 74 at any integer head position. PA proves the corresponding coded finite-trace equivalence, in particular for the blank start. The arithmetical statement ranges over numerically coded configurations and finite traces; the arbitrary-complete-tape formulation is external, and its local proof allows every outside-tape extension.*

Proof. Let σ be the displayed quotient map. For each pair (a, b), every incoming transition to a writes zero and moves right; every incoming transition to b writes one and moves right. Neither is initial. Induction on a parent trace therefore gives $t(h-1) = 0$ at a and $t(h-1) = 1$ at b. The checker verifies all 22 incoming rows establishing these premises.

For each of the 14 controls and both current bits, the final simultaneously folded table reproduces the parent transition's write, head displacement, and projected successor. All reads and writes in these 28 cases remain in the two cells $h-1, h$, and the entire endpoint tape agrees. The durations are ten cases of length one, seventeen of length three, and one of length five. Every other transition commutes with σ in one step. Hence each reachable parent step is simulated by one to five child steps.

Concatenation gives forward halting preservation. Conversely, if the child halts at time T but the parent has not halted within T steps, its first $T+1$ defined steps concatenate to a child run longer than T. All blocks are positive and their intermediate controls are nonhalting. Determinism contradicts the asserted halt. This is a bounded finite-trace argument; its lookup, invariant, and concatenation functions are primitive recursive and its inductions are available in PA. □

4 The unchanged numerical calculus

Using the Cantor pairing p, formulas are numerical codes with constructors

$$I(A, B) = 4p(A, B), \quad M(a, b) = 4p(a, b) + 1, \quad U(a, B) = 4p(B, a) + 2, \quad E(a, b) = 4p(a, b) + 3.$$

Zero is truth and $1 = M(0, 0)$ is the designated false code. Write $A \to B$ for $I(A, B)$, $\neg A = I(A, 1)$, and use the classical abbreviations for conjunction and $\exists x_a$. The six retained heads are the following universally parameterized numerical schemas.

There is also the dominated propositional schema

$$\rho_8(D, A, H, u) = ((H \to u) \to D) \to ((D \to H) \to (A \to H)), \qquad u \le D,$$

and the guarded diagonal bundle explicitly defined in Section 5. The inference rules are modus ponens and generalization of a theorem at any numerical label. No extra equality rule, truth oracle, or arbitrary-tautology rule is implicit in this specification. The source verifier proves the complete emitted stack and gate as identities in a free algebra, not by testing small numerical instances.

Head	Formula
1	$(\exists x_D\, U(D,H)) \to H$
2	$U(D,U(A,H)) \to U(A,U(D,H))$
3	$M(H,D) \to (E(H,A) \to M(A,D))$
4	$E(H,D) \to (M(A,H) \to M(A,D))$
5	$\exists x_D\, E(H,D)$
6	$U(D,A) \to (U(D,A \to H) \to U(D,H))$

Table 1: The six unchanged numerical heads.

5 Arithmetic calibration of the unchanged diagonal calculus

This section reproduces the mathematical calibration from the original PA195 report [3], with its machine-specific opening and original command analysis omitted. It is inherited mathematics, included to make the logical strength of the new table explicit. The new machine uses exactly this calculus. The changed command compiler is proved separately in Section 6. All statements here are ordinary mathematical proofs of PA-provability claims; no new kernel-checked endpoint is asserted.

5.1 The numerical calculus and its diagonal replacement

Write

$$p(a,b) = \frac{(a+b)(a+b+1)}{2} + b, \quad I(A,B) = 4p(A,B), \quad M(a,b) = 4p(a,b)+1,$$
$$U(a,B) = 4p(B,a)+2, \qquad E(a,b) = 4p(a,b)+3.$$

The notational convention is $\neg A = I(A,1)$, conjunction and existential quantification being their classical abbreviations. Code 0 is truth, and code $1 = M(0,0)$ is the designated contradiction. The quantifier constructor has the transposed Cantor arguments displayed above.

The diagonal calculus $\mathcal{C}_\Gamma$ retains the six PA196 heads, the dominated propositional head, modus ponens, and closed generalization. Its guarded gate is

$$\begin{aligned}
\Gamma(B) &= I(B,U(B,B)),\\
V(a,b,c) &= I(M(a,b),U(c,M(a,b))),\\
P(a,b,c) &= I(c,M(c,b)),\\
S(a,b,c) &= U\big(b,(M(a,b) \to P(a,b,c)) \wedge (M(c,b) \to c)\big),\\
Q(a,b,c) &= U\big(c,S(a,b,c) \to \neg P(a,b,c)\big),\\
F_\Gamma(a,b,c) &= \neg\big(\Gamma(c) \to (V(a,b,c) \to Q(a,b,c))\big).
\end{aligned}$$

The gate is available when $c \neq a,b$; a failed guard emits 0. The logical rules and remaining axiom families are exactly those of `PA196/Logic/Basic.lean`, with this one replacement of its `bundle`.

Every recursive formula child of a nonzero code is smaller than that code. Every variable label, including a bound label, occurring in a formula of code B is strictly below B. These assertions follow from the inequalities $a,b \le p(a,b)$ and the four tags, with code zero handled separately. Consequently x_B is absent from the decoded formula B. The diagonal clause is a valid vacuity instance, although its binder is chosen from the numerical code of the entire formula.

Classical implicational reasoning extracts from the gate both $\Gamma(c)$ and the old gate $G(a,b,c) = \neg(V(a,b,c) \to Q(a,b,c))$. Every diagonal instance $\Gamma(B)$ is available: use $c = B$ and $a = b = B+1$. Thus all membership-vacuity instances and pointed separation used in the original proof remain available.

Lemma 5.1 (Recovery of the required equality interfaces)**.** *The diagonal calculus derives, uniformly in the numerical labels,*

$$E(a,b) \to \big(M(d,a) \leftrightarrow M(d,b)\big),$$
$$E(a,b) \to \big(M(a,d) \leftrightarrow M(b,d)\big),$$
$$E(a,b) \to R(a,b,z) \quad (z \neq a,b),$$

where

$$R(a,b,z) = U\big(z, M(a,z) \leftrightarrow M(b,z)\big) \wedge U\big(z, M(z,a) \leftrightarrow M(z,b)\big),$$

and it derives $\exists x_a R(a,b,z)$ for $z \neq a,b$. All these derivations are supplied by primitive-recursive proof templates, with correctness provable in PA.

Proof. The two forward membership rewrites are retained heads. To obtain the reverse directions, first assume $a \neq b$. Take $\chi(t) = M(d,t)$ or $M(t,d)$, initially with $d \neq a$, or take the diagonal family $\chi(t) = M(t,t)$. Put

$$C = E(a,b) \to (\chi(b) \to \chi(a)), \qquad \varphi = \neg C, \qquad c = \ulcorner \varphi \urcorner.$$

Here c is the actual numerical code of the displayed formula, so it is larger than every label in C. In particular, all required guards hold. Apply the old gate at (a,a,c) and abbreviate its S, P, Q by those letters. We need only its consequence $\neg Q$, not an unproved general rule for changing bound-variable names.

The second conjunct in S gives $S \to U(a, M(c,a) \to \varphi)$. Propositionally, φ implies $E(a,b)$, $\chi(b)$, and $\neg\chi(a)$; hence universal monotonicity gives

$$S \to U(a, M(c,a) \to \neg\chi(a)).$$

The equality atom has disappeared from this quantified consequent. The unchanged negative-property specialization lemma now gives

$$S \to \big(M(c,b) \to \neg\chi(b)\big). \tag{1}$$

For precision, this specialization uses only the retained witness $\exists x_a E(b,a)$, direct rewriting under $E(b,a)$, and vacuity of $M(c,b) \to \neg\chi(b)$ at a. The latter follows from membership vacuity because $a \neq b, c$ and, in the two nondiagonal families, $a \neq d$. It is exactly the construction `negativeSpecialization` in `PA196/Logic/Reverse.lean`; none of its premises is primitive equality vacuity.

Under φ, the point condition P gives $M(c,a)$. Direct element-slot rewriting under $E(a,b)$ then gives $M(c,b)$. Together with (1), S contradicts the consequence $\chi(b)$ of φ. Classical deduction gives $\varphi \to (S \to \neg P)$. Generalize this theorem at c, distribute the quantifier through implication, and use $\Gamma(\varphi) = \varphi \to U(c,\varphi)$. The result is $\varphi \to Q$. Since the gate supplies $\neg Q$, double-negation elimination gives C, as required.

If $a = b$, the reverse rewrite is propositional. When the fixed label $d = a$, direct rewriting first takes $M(a,b)$ or $M(b,a)$ to $M(b,b)$; the already proved diagonal case takes $M(b,b)$ to $M(a,a)$. This covers every label coincidence without a freshness assumption on the theorem being derived.

For the neighborhood formula choose $z_0 = \ulcorner E(a,b) \urcorner$. The available diagonal clause $E(a,b) \to U(z_0, E(a,b))$ and the two bidirectional rewrites yield $E(a,b) \to R(a,b,z_0)$. To change the binder to an arbitrary $z \neq a,b$, only the case $z \neq z_0$ remains. Let $P(t)$ be either membership-only matrix in R. Bidirectional rewriting gives $E(z,z_0) \to (P(z_0) \to P(z))$. The witness $\exists x_{z_0} E(z,z_0)$, universal distribution, and vacuity of $P(z)$ at z_0 give

$$U(z_0, P(z_0)) \to P(z).$$

Its antecedent is vacuous at z: its nonconstant membership atoms have indices only a, b, z_0. The designated false code $1 = M(0,0)$, which occurs in the Boolean abbreviations, is vacuous at every label by explosion. Vacuity extends through the Boolean connectives and the outer quantifier by the retained commutation head. Generalization and distribution therefore give $U(z_0, P(z_0)) \to U(z, P(z))$. This proves the claimed neighborhood implication for every allowed z. Finally the unchanged witness head and existential monotonicity give the neighborhood witness, reversing its two arguments by the propositional symmetry of R if necessary.

Every operation in this proof is a fixed syntactic proof template, composition, numerical code formation, or a displayed decidable label-equality case split. The choices $c = \ulcorner \neg C \urcorner$ and $z_0 = \ulcorner E(a,b) \urcorner$ use no unbounded search. PA verifies the templates by the same proof-checker induction as their original counterparts. This establishes the asserted uniformity. □

The PA196 refutation compiler uses primitive equality through these membership-rewriting, neighborhood, and witness interfaces. In its logical and arithmetic certificate copies, the uses of equality vacuity are in the reverse-rewriting and neighborhood modules just replaced. The old gate, all other heads, and the command stack interface remain available. Substituting Lemma 5.1's templates therefore gives a PA-verifiable refutation compiler for $\mathcal{C}_\Gamma$. In particular,

$$\mathrm{PA} \vdash \exists p \, \mathrm{Prf}_{\mathrm{PA}}(p, \ulcorner 0 = 1 \urcorner) \;\longrightarrow\; \exists d \, \mathrm{Prf}_{\Gamma}(d, 1). \tag{1}$$

This is reuse of the published proof-producing construction with explicit replacement templates, rather than an appeal to standard-model equiconsistency. The 191-item AX word still pushes nine items in the same positions as the original nine-item command, so its stack compiler indices need no alteration.

5.2 A uniform translation of the numerical calculus into PA

We now prove the converse proof-theoretic direction. Work first in the fixed conservative definitional extension of PA containing $\mathrm{pow2}(n) = 2^n$ and its usual arithmetical definition. Only fixed primitive-recursive functions are added. A fixed primitive-recursive elimination of these definitions converts the resulting proofs to ordinary PA proofs.

Use the bit predicate

$$B(s,z) \;\equiv\; \exists u, v \, [s = (2u+1)2^z + v \;\wedge\; v < 2^z].$$

PA proves the usual digit lemmas directly by induction and division with remainder. In particular $B(s,z) \to z < s$, since $s \geq 2^z \geq z+1$, and hence PA proves $\neg B(x,x)$.

Let $T(c)$ be the arithmetic formula obtained by decoding the source code c:

$$T(0) = (0=0), \qquad T(M(a,b)) = B(x_a, x_b),$$
$$T(E(a,b)) = (x_a = x_b), \qquad T(U(a,A)) = \forall x_a \, T(A).$$

For a nonzero implication code the translation is the corresponding implication. There is one numerical alias: $I(0,0) = 0$. Thus the uniform constructor theorem is a proof of

$$T(I(A,B)) \leftrightarrow (T(A) \to T(B)),$$

not an unconditional assertion of syntactic identity. At $A = B = 0$ it uses the fixed proof $(0=0) \leftrightarrow ((0=0) \to (0=0))$. Every other case follows from the definition of T.

Arithmetic auxiliary bound variables are represented with de Bruijn indices, or equivalently renamed above all source labels before substitution. The source label c is therefore absent among the free variables of $T(c)$ even though the expanded bit formula has its own bound variables. Quantifier formation closes the indicated free source label, so nested occurrences of the same label have their usual shadowing semantics. No assumption that all numerical labels are distinct is made.

Lemma 5.2 (Uniform finite bit comprehension)**.** *Given the syntax code of any arithmetic formula $\theta(z, \vec{u})$, there is a primitive-recursive procedure producing a PA proof of*

$$\forall n\, \exists s < 2^n\, \forall z < n\, \big(B(s, z) \leftrightarrow \theta(z, \vec{u})\big). \tag{2}$$

PA proves that the procedure produces a valid proof for every well-formed formula input, with the indicated fresh variables.

Proof. Take induction on n with the existential formula in (2) as induction formula. At zero choose $s = 0$. Given a witness $s < 2^n$, split by excluded middle on $\theta(n, \vec{u})$. If it holds use $s + 2^n$; otherwise use s. The fixed bit lemmas say that the first n bits are unchanged, that the new nth bit is respectively one or zero, and that the result is below 2^{n+1}. This is a fixed finite first-order proof pattern with one PA induction instance.

On syntax codes the procedure performs capture-avoiding substitution, forms the displayed induction instance, and concatenates this fixed proof pattern with the fixed arithmetic lemma proofs. It never evaluates θ. Its syntax operations and a fresh-bound-variable choice are primitive recursive. The PA proof of correctness is induction on the syntax operations, followed by direct verification that the generated induction line is an instance of the PA induction schema and that each subsequent line follows by the stated logical rule. Thus this is a statement about proof checking, with an arbitrary formula code as input, not a uniform satisfaction or reflection principle for PA. Full PA induction is used: θ is not restricted to a bounded or quantifier-free formula. □

We spell out the gate supplier because label collisions matter here. Fix a, b, c with $c \neq a, b$, let $\phi = T(c)$, and regard x_a, x_b as the free values before the inner quantifier is entered. If $a \neq b$, apply (2) with bound $x_a + x_b + 1$ and matrix

$$\big(B(x_a, z) \vee z = x_b\big) \wedge \phi[z/x_b]. \tag{3}$$

The witness s satisfies, for every z,

$$B(x_a, z) \to (\phi[z/x_b] \to B(s, z)), \qquad B(s, z) \to \phi[z/x_b],$$

and $\phi \to B(s, x_b)$. To pass from the bounded comprehension formula to the displayed unbounded assertions, use $B(x_a, z) \to z < x_a$ and the fact that a number below 2^n has no set bit at an index at least n.

If the *labels* $a = b$ coincide, the first antecedent inside $T(S)$ becomes $B(z, z)$, not $B(x_a, z)$ with x_a held fixed. It is therefore false. Apply (2) instead with bound $x_b + 1$ and matrix

$$(z = x_b) \wedge \phi[z/x_b]. \tag{4}$$

This witness has the required second implication and distinguished point condition, and the first implication is vacuous. Equality substitution in (4) supplies ϕ at the distinguished point. The substitutions in (3)–(4) are capture avoiding; in particular the inner binder corresponding to b is renamed when the witness expression depends on the original free value of x_b.

In both cases the result is a PA proof of $\exists x_c\, (T(S) \wedge T(P))$. Freshness of c in ϕ ensures that choosing the set witness does not alter the property. Consequently PA proves $\neg T(Q)$. The guarded membership-vacuity clause has a first-order proof, and $T(\Gamma(c))$ has a first-order proof by the code-freshness lemma. Classical propositional reasoning therefore proves $T(F_\Gamma(a, b, c))$. When the guard fails the command emits 0, with its fixed truth proof.

The source false code is translated to $T(1) = B(x_0, x_0)$. Its negation is a fixed PA theorem, uniformly under all binders. Thus source negation, conjunction, and existential abbreviations have PA-provable

equivalences to the ordinary classical connectives, even if a surrounding source quantifier binds label zero. This deals with the false-code convention independently of the truth-code alias.

The other head suppliers are fixed first-order proof templates: the witness-elimination head, quantifier commutation, the two equality congruence heads, an equality witness, and quantifier distribution. Coincident labels are handled by capture-avoiding instantiation, with a numerical equality case split if needed. The dominated propositional head is a classical tautology; its arithmetic validity does not require the dominance restriction. Generalization and modus ponens use their ordinary PA proof constructors, together with the implication-constructor equivalence above.

Lemma 5.3 (Syntactic refutation translation)**.** *There is a primitive-recursive function J such that*

$$\mathrm{PA} \vdash \forall d \, (\mathrm{Prf}_{\Gamma}(d, 1) \to \mathrm{Prf}_{\mathrm{PA}}(J(d), \ulcorner 0 = 1 \urcorner)) \,. \tag{5}$$

Proof. Decode the source derivation by recursion on its finite proof code. At an axiom node insert the appropriate supplier just described. At modus ponens concatenate the two translated proofs and the compositional implication proof; at generalization append the corresponding quantifier rule. At a code-zero alias emit or use the fixed truth equivalence. Recursion on a malformed proof is made total by an arbitrary fixed output; the correctness statement has the displayed proof-validity hypothesis.

PA proves the syntactic invariant $\mathrm{Prf}_{\Gamma}(d, A) \to \mathrm{Prf}_{\mathrm{PA}}(J_0(d), T(A))$ by induction on this coded recursion. Constructor children decrease strictly; proof concatenation, label closing and fresh-variable renaming have primitive-recursive definitions and their usual syntactic correctness inductions. The gate case is precisely Lemma 5.2 and the two label cases above. Append the fixed theorem $\neg B(x_0, x_0)$ at a proof of code one, then eliminate the fixed definitional extension. This gives (5).

This argument proves one quantified assertion about proof codes inside PA. It does not merely produce a PA proof separately for each standard formula and then assert a uniform conclusion. The inductive verification of the compiler, including the schema supplier, is the step that makes (5) applicable to nonstandard codes in every model of PA. □

6 Why the changed controller still enumerates refutations

This section specializes the already supplied Combined command argument [5, 6, 7] to the nine-entry diagonal emitter. The controller transformation is inherited; its composition with the seven-fold 195-state table is the new construction. Explicit contracts are needed: changing where modus ponens leaves its result also changes the proof compiler.

Let $p(a, b) = \binom{a+b+1}{2} + b$ and $[a; w] = p(a, w)$. Cantor decoding is total and satisfies

$$\mathrm{hd}[a; w] = a, \qquad \mathrm{tl}[a; w] = w, \qquad [0; 0] = 0.$$

The last identity must be retained. A numerical stack is not a free list with a distinct empty constructor. Iterated cons is written $[a_1, \ldots, a_k; w]$, with a_1 on top. Write a command store as $\langle t, w, d, h \rangle$, where t is the top formula, w the underlying stack, d an offset, and h the cached formula. Put $I(a, b) = 4p(a, b)$.

6.1 Exact new commands

The new Shift command is

$$S\langle t, w, d, h \rangle = \begin{cases} \mathsf{halt}, & h = 1, \\ \langle h, \mathrm{tl}(w), d, \mathrm{hd}(w) \rangle, & h \neq 1. \end{cases}$$

For $y = d + x$, the new modus-ponens command is

$$\mathsf{MP}_x\langle t, w, d, h\rangle = \begin{cases} \langle 0, w, 0, y\rangle, & I(t, y) = h, \\ \langle 0, w, y, 0\rangle, & I(t, y) \neq h. \end{cases} \tag{C1}$$

Only Shift can halt. Successful MP puts the conclusion in the cache and clears the offset. It does not push a dummy entry. The old controller instead returned $\langle y, [0; w], y, 0\rangle$ on success and tested $t + h = 1$ at Shift. Thus the two controllers have different store transitions, even though both recognize refutability of the same calculus.

For $D = d + x$, $H = h + y$, $A = c$, the unchanged AX command has the exact effect

$$\mathsf{AX}_{c,x,y}\langle t, w, d, h\rangle = \langle K(H, D, c), [q_1, \ldots, q_8, t; w], 0, 0\rangle, \tag{C2}$$

where

$$(q_1, \ldots, q_8) = (L_6, L_5, L_4, L_3, L_2, L_1, \rho_8(D, A, H, d), U(d, h)).$$

Here $L_1, \ldots, L_6$ are the six heads in Table 1, and K is the guarded diagonal gate of Section 5, with zero on guard failure. All generated values except $U(d, h)$ are implication codes and have residue zero modulo four; $U(d, h)$ has residue two. Consequently none of $q_1, \ldots, q_8, K$ is the false code one. These are numeric residue facts, independent of any consistency assumption.

6.2 Uniform stack operations

Commands in a word execute from left to right. For every incoming store, including $t = 1$ and every Cantor zero alias, (C2) gives

$$\mathsf{AX}_{c,x,y}; S^8 : \langle t, w, d, h\rangle \longmapsto \langle \rho_8(d + x, c, h + y, d), [t; w], 0, U(d, h)\rangle,$$
$$\mathsf{AX}_{c,x,y}; S^9 : \langle t, w, d, h\rangle \longmapsto \langle U(d, h), w, 0, t\rangle.$$

The first Shift tests zero and the next eight test generated entries; none of these nine tests examines the saved top t.

Define the following concrete words:

$$\begin{aligned} \mathsf{Push} &= \mathsf{AX}_{0,0,0}; S^8; \mathsf{MP}_0, \\ \mathsf{Pop} &= \mathsf{AX}_{0,0,0}; S^{10}, \\ \mathsf{Discard} &= \mathsf{MP}_1; \mathsf{AX}_{1,0,0}; S^9; \mathsf{MP}_0. \end{aligned}$$

Their contracts are

$$\begin{aligned} \mathsf{Push}\langle t, w, d, h\rangle &= \langle 0, [t; w], 0, 0\rangle, \\ \mathsf{Pop}\langle t, [a; w], d, h\rangle &= \begin{cases} \mathsf{halt}, & t = 1, \\ \langle t, w, 0, a\rangle, & t \neq 1, \end{cases} \\ \mathsf{Discard}\langle t, w, 0, 0\rangle &= \langle 0, w, 0, 0\rangle. \end{aligned}$$

For Push, its last comparison is between an implication and a universal code, so MP fails with offset zero. Pop's tenth Shift tests the saved top and, when safe, removes the desired underlying entry. For Discard, $I(t, 1) > 0$ forces the first MP to fail, producing offset one. The AX identity then gives $\langle U(1, 0), w, 0, 0\rangle$, and $I(U(1, 0), 0) > 0$ forces the final MP to fail with offset zero. Pop is deliberately partial at a derived contradiction; the proof compiler accounts for this early halt.

Let Offset_0 be empty and let $\mathsf{Offset}_i = \mathsf{MP}_i$ for $i > 0$. Starting at $\langle 0, w, 0, 0\rangle$, it produces $\langle 0, w, i, 0\rangle$. The word

$$\mathsf{Gen}_i = \mathsf{Push}; \mathsf{Offset}_i; S; \mathsf{AX}_{i,0,0}; S^9$$

sends $\langle a, w, 0, 0\rangle$ to $\langle U(i, a), w, 0, 0\rangle$ for every a, w. AX clears the cache before it could be tested, so the contract holds even when $a = 1$.

6.3 Axiom selectors and modus ponens

For $1 \le j \le 6$, an ordinary head selector is

$$\mathsf{Discard}; \mathsf{AX}_{A,D,H}; S^{j+1}; \mathsf{Pop}^{8-j}.$$

It returns $\langle q_j, w, 0, 0\rangle$ from $\langle t, w, 0, 0\rangle$. The remaining displayed stack entries, ending in the saved zero, are consumed by the Pops. Each preserves $q_j \neq 1$. For $u \le a$, the restricted propositional head is selected by

$$\mathsf{Discard}; \mathsf{Offset}_u; \mathsf{AX}_{b,a-u,c}; S^8; \mathsf{Pop}.$$

This returns $\rho_8(a, b, c, u)$. The gate selector $\mathsf{Discard}; \mathsf{AX}_{c,b,a}; \mathsf{Pop}^9$ returns $K(a, b, c)$. All preserve the arbitrary underlying stack and leave both caches zero; their generated tops have residue zero.

The exact MP suffix is

$$\mathsf{MPSuffix}_b = S; S; \mathsf{MP}_b; S.$$

On the store $\langle 0, [a, I(a, b), 0; w], 0, 0\rangle$, it halts if $a = 1$ or $b = 1$, and otherwise returns $\langle b, w, 0, 0\rangle$. The first Shift fetches a; the second tests it and fetches the major premise. MP succeeds and puts b in the cache. The final Shift tests the conclusion and consumes the dummy zero. Without that zero the suffix would consume an entry of the arbitrary underlying stack. At $a = b = w = 0$, total Cantor decoding proves exactly the same identities.

Lemma 6.1 (Uniform proof-word compiler)**.** *For every finite proof tree p in $\mathcal{C}_\Gamma$, a primitive-recursively constructed word W_p, from every $\langle t, w, 0, 0\rangle$, either halts or returns $\langle \mathrm{concl}(p), w, 0, 0\rangle$. If p proves one, a word constructed from p halts from every incoming store, including arbitrary nonzero caches.*

Proof. Compile axiom leaves by the preceding selectors and generalization by $W_{p_0}; \mathsf{Gen}_i$. For an MP node with minor premise p_0 proving a and major premise p_1 proving $I(a, b)$, use

$$W_p = \mathsf{Discard}; \mathsf{Push}; W_{p_1}; \mathsf{Push}; W_{p_0}; \mathsf{Push}; \mathsf{MPSuffix}_b.$$

The first two words reserve a dummy zero. Induction on p supplies the two premises above it, with the original stack framed. The suffix has precisely its required precondition. Any earlier halt remains a halt after extending the word. This proves the first assertion. If the conclusion is one, use $\mathsf{AX}_{0,0,0}; W_p; \mathsf{Pop}$. The initial AX clears both caches from every incoming store; if W_p has not already halted, the last Pop tests its conclusion before inspecting the underlying stack. □

The unchanged scheduler exhaustively enumerates finite command words. The universal incoming-store property matters: failed earlier trials may leave a store behind. The successful word need not begin with an empty stack. The inherited fetch and operand-decoding proofs therefore apply with the modified command contracts, as their source prefix and all resolved successors are checked explicitly.

6.4 Soundness as a finite proof annotation

Maintain derivations of t, h, and every canonical stack entry in $\mathcal{C}_\Gamma$; impose no derivability condition on d. For $w > 0$, $\mathrm{tl}(w) < w$, so iterating the tail reaches zero in at most w steps. This defines a primitive-recursive finite canonical stack. The exceptional identity $[0;0] = 0$ is handled by the fixed proof $\rho_8(0,0,0,0) = 0$: popping the implicit zero tail always supplies that proof and requires no infinite annotation.

AX preserves the annotation: it saves t, generalizes the old h, emits axioms, and respects $d \le d + x$. MP failure returns zero, while success applies modus ponens to the annotated t and $h = I(t, d + x)$. Shift moves annotated entries; a halting Shift exposes an annotated one. These proof updates are primitive recursive. Induction on a finite command execution proves that a halt yields a calculus refutation. Conversely, Lemma 6.1 and exhaustive scheduling turn a calculus refutation into a finite source halt. Both are PA-verifiable syntactic constructions. Together with Section 5,

$$\mathrm{PA} \vdash \mathrm{Halt}(G_{\mathrm{DC}}, \mathrm{seed}) \leftrightarrow \exists p \; \mathrm{Prf}_{\mathrm{PA}}(p, \ulcorner 0 = 1 \urcorner). \tag{C3}$$

Here G_{DC} is the explicitly delivered 3,676-instruction register program. The next section binds its changed arithmetic loops and its literal binary-tape implementation.

7 Binding the proof to the literal binary machine

7.1 The changed source loops

The logical register names are $T = R_1$, $W = R_2$, $D = R_4$, $H = R_5$, with scratch $X = R_6$, $Y = R_7$. The command-boundary invariant includes $R_3 = X = Y = 0$. It leaves the four store values arbitrary. Candidate register R_0 and the remaining command word R_8 are preserved except where the inherited scheduler and operand decoder explicitly use them.

Here is the complete changed tail. A decrement lists the zero successor first and the positive successor second; on the positive branch it decrements the named register.

Ordinal	Resolved instruction
3661	$\mathrm{dec}(D, 3664, 3662)$
3662	$\mathrm{inc}(H, 3663)$
3663	$\mathrm{jump}(3661)$
3664	$\mathrm{jump}(25)$
3665	$\mathrm{dec}(T, 3667, 3666)$
3666	$\mathrm{jump}(3665)$
3667	$\mathrm{dec}(H, 3671, 3668)$
3668	$\mathrm{inc}(T, 3669)$
3669	$\mathrm{inc}(X, 3670)$
3670	$\mathrm{jump}(3667)$
3671	$\mathrm{dec}(X, 3674, 3672)$
3672	$\mathrm{inc}(H, 3673)$
3673	$\mathrm{jump}(3671)$
3674	$\mathrm{inc}(X, 3675)$
3675	$\mathrm{jump}(0)$

On a successful MP, the unchanged prelude supplies $(T, W, D, H, X, Y) = (0, w, y, 0, 0, 0)$ at 3661. After k loop iterations, $(D, H) = (y - k, k)$. The loop reaches 3664 in $3y + 1$ source steps and returns to fetch in one more. A failed comparison goes directly to 3664 with offset y and zero cache. This proves (C1) for arbitrary naturals.

At Shift entry 3665, the first loop clears $T = t$ in $2t + 1$ instructions. The next loop transfers $H = h$ into both T and X in $4h + 1$ steps, and the third restores H from X in $3h + 1$ steps. Thus the first nine tail instructions, iterated as prescribed, reach $(T, H, X) = (h, h, 0)$ after $2t + 7h + 3$ steps. The final two rows set $X = 1$ and enter the unchanged root guard, now comparing h with one. On a nonhalting branch, the unchanged Cantor decoder produces $\mathrm{hd}(w)$ and $\mathrm{tl}(w)$. The elementary loop invariants give precisely the new Shift contract for all store values. They do not rely on tests of a bounded initial computation.

The checker verifies all unchanged prefix rows and the three exit relocations before checking this tail. The source therefore has the command semantics proved in Section 6. Its census is 1,458 increments, 1,108 decrements, 1,109 resolved jumps, and one halt. Counting both decrement outcomes gives

$$1458 + 2(1108) + 1109 + 1 = 4784.$$

7.2 Clean tape frames and two independent checks

A clean frame has a left marker, a binary source address, a zero-filled separator, a right header, and a complete unary register file. A value v uses the block $0\,1^{v+1}$. Logical registers $0, \ldots, 8$ occupy physical positions

$$(6, 2, 0, 4, 3, 7, 8, 5, 1).$$

The header distance is $48 + n$ for arbitrary $n \geq 0$; the longest source path here has 38 bits. The physical file has the nine logical registers and an arbitrary finite nonempty spare suffix. Increment appends a one-valued spare block, as in the inherited register engine. That effect is part of the contract, rather than silently ignored.

The primary checker reconstructs original PA195 from its supplied grammar, folds, and pinned vendor files, then reconstructs PA195DC. It checks the source graph with two differently structured resolvers, the complete AX free-algebra output, unchanged arithmetic-box rows, every instruction outcome for the unfolded and final folded tables, and full initialization. Its symbolic execution uses the inherited arbitrary-register block contracts. It accepts 4,784 contracts for each table; the folded run has 1,573,300 symbolic steps in total and a maximum of 506 per contract.

The independent checker reads the literal final table and the delivered source, without importing the primary compiler. It reconstructs the source graph independently and rederives four generic register-list sweep lemmas from that table. It then follows all source instructions with symbolic natural-valued unary blocks and a symbolic finite spare list. It does not call an arithmetic-box oracle. The resulting verification is:

Independent physical check	Result
Source instructions / outcomes	3,676 / 4,784
Total symbolic steps	1,152,450
Largest symbolic step count	409
Literal / unary-scan / list events	977,255 / 172,279 / 2,916
List lemmas derived from the literal table	4
Rows occurring outside list-summary applications	390 of 390
Arithmetic-box oracles	None
Exact blank startup	57,922 transitions

These are different symbolic granularities, so their step totals need not agree. Neither maximum is a bound on actual tape steps for arbitrary register values. The symbolic formulas describe all natural

register values and finite spare lists; the counts enumerate proof cases, not small numeric samples. The symbolic reasoning is executable mathematics and is not asserted to have been newly checked by a proof-assistant kernel.

7.3 Exact initialization

Literal execution from the all-zero tape reaches the clean source-25 frame after exactly 57,922 transitions, in native state 84 with head at -2. The new source path is `1000000`; the logical register vector is $(2, 0, 0, 0, 0, 0, 0, 0)$, and the spare suffix has 26 registers of value one. The header distance is 48. Both checkers compare the entire finite support of the tape, the state, and the head, and require every other cell to be zero. The certificate retains the complete sorted set of one positions. The original PA195 startup took 58,154 transitions; the saving is 232 transitions in initialization. It is not a claim of uniform runtime improvement throughout the infinite search.

7.4 Uniform finite-trace transfer inside PA

Each symbolic instruction contract denotes a finite, positive literal execution segment from a clean source frame to the exact next frame, or to halt for the single source halt instruction. Symbolic unary scans have natural-valued lengths; list traversals use induction on the finite register list. Their trace constructors and duration bounds are primitive recursive in the coded frame. The unchanged header and spare invariants are preserved. The arithmetic loops above and the scheduler have analogous primitive-recursive finite trace constructors.

For clarity, configurations can be coded by a control and two binary stacks for the tape on either side of the head. A finite run is a coded finite sequence checked by the fixed transition function. PA proves the usual coding identities, the scan and list inductions, and the finite symbolic identities. Since the table and the contract list are fixed finite objects, their case distinctions require no additional schema beyond PA induction. This internal argument concerns all coded finite traces, including model-length traces. An external Lean theorem only about standard finite runs would not by itself justify that step.

Lemma 7.1 (Literal/source halting equivalence)**.** *PA proves that M_{DC} halts from blank if and only if the delivered source program G_{DC} halts from the exact clean seed.*

Proof. Forward source-to-tape transfer concatenates the fixed startup and the positive physical segments of a finite source trace. All constructed runs and their bounds are primitive recursive, and PA verifies them by induction.

For the converse, suppose the literal machine halts at time T. If it halted before the end of startup, the checked finite startup would already contradict determinism. Otherwise start at the exact seed and suppose the source does not halt within its first $T + 1$ instructions. Iterating those instructions and their contracts constructs a defined physical continuation longer than T, with no premature halt. Determinism contradicts the alleged halting trace. Thus the source halts within that bound. This does not assume that a physical tape can be inversely decoded at arbitrary intermediate times, or that the machine actually runs forever. Positivity of the segments is the essential bounded argument. □

The dense and native tables have a separately checked row-for-row conjugacy. Mapping each control of a finite trace along that fixed bijection gives one-step exact conjugacy and fixes the specified start/halt pair. This finite induction also formalizes in PA, completing the binding to the definitive dense table rather than merely to inherited native labels.

Proof of Theorem 1.1. The uniform proof translations of Section 5 identify refutability of $\mathcal{C}_\Gamma$ with refutability of PA, internally in PA. Section 6 identifies the former with source halting, using both the annotation invariant and the complete proof-word compiler. The exact source binding and Lemma 7.1 identify that source halt with a blank halt of the literal table. Compose the three internal equivalences and negate them in classical PA. This gives the stated biconditional with Con(PA). □

8 The search below 195 and what the negative results mean

No verified 194-state machine was obtained. The result is the user's same-size alternative, with a source-level controller change and a new literal graph. The comparison and search receipts support this statement; they do not establish a lower bound of 195 or exhaust all possible two-symbol machines.

8.1 Finite-window local reductions

We computed an inductive overapproximation of the head-centered tape windows. A transition overwrites the central bit, shifts the window one cell, and exposes either possible bit at its far edge. Starting from the zero window is sound for the blank computation: introducing both possible new edge bits only enlarges the represented set. A least fixed point on this finite abstract graph supplies all allowed local contexts. The radius-seven runs used 15-cell windows and gave:

Table	Reachable abstract contexts
Original diagonal PA195	922,054
Separate-head 195S	931,579
New PA195DC	916,710

For each, we checked 37,636 ordered direct identifications retaining the keeper's rows, with a noninitial removed state. We also checked all 18,721 unordered pairs of noninitial controls in a two-rewritten-row template: the keeper's first step can write either bit, move either way, and enter an existing helper. Each local simulation must reproduce one parent transition within 63 steps, stay inside the recorded window, restore the exact tape, and reach the projected state and head. Helper runs reject an interior revisit to the merged keeper before the required endpoint. Both possible starting read bits and all abstract contexts are checked. The PA195DC run attempted 14,631,480 row actions after early rejection of infeasible first-row choices. No pair passed this template.

As a positive control, the same program on the unfolded 202-state diagonal parent found all seven known fold pairs. The full compatibility of the retained simultaneous fold is verified separately by the 28-case certificate. Radius-five arbitrary-initial-tape runs provide additional recorded controls. The search's limited template and 63-step cutoff are explicit; no failure is promoted to a universal state-minimality theorem.

8.2 Program trees and alternative logical emitters

The fold-aware grammar searches score the actual allocation of readers after compatible folds, rather than merely minimizing source-tree nodes. They explored binary reassociation, dynamic permutations of the six pure axiom-head blocks, local parameter permutations, antecedent exchange in three implicational heads, and a permutation of the two unrestricted parameters in the dominated propositional head. Additional searches allowed fold candidates involving the fixed core and used the new Combined core,

as well as two longer dual-diagonal clauses. The five corrected runs proposed 39,661,078 moves and accepted 7,153,880; none produced a sub-195 candidate. A final radius-two rotation neighborhood contained 49,513 distinct endpoint trees and also found none. The retained source/input files and fixed-seed logs state the finite budgets and tested fold model. Two secondary 195-state witnesses passed full finite implementation checks; their additional logical calibration is separate from the primary result. The selected machine needs none of these stochastic searches for reproduction: its exact witness is frozen and deterministic.

Appendix C proves the head-order change safe at the command interface. Appendix D records a useful obstruction. Replacing the equality-witness head $\exists x_D E(H, D)$ by the shorter reflexivity $E(H, H)$ does not preserve the needed axiom interface: a variable-dependent-domain model validates the other proposed axioms while refuting the witness. This rules out a simple reuse of the old completeness proof. It does not decide the consistency strength of the altered theory.

8.3 What was useful from ZFC and ConciseIPRA

The ZFC lineage supplies the unary-register engine, shared source readers, and history-sensitive use of the adjacent tape cell [10]. Those are already central to the PA lineage. Transferring an optimized row from a different encoding requires preserving its entry conditions; the corpus contains failed examples of a blind zero-spare transplant. Here the full table and its frame contracts were checked after composition.

ConciseIPRA emphasizes explicit syntax, proof checking, and economical proof-theoretic interfaces [11]. The diagonal formula-code freshness argument is useful in that spirit. The PA calibration, however, needs finite bit comprehension for an *arbitrary* translated formula, proved using full PA induction. A short iPRA checker cannot replace that strength without a new equivalence proof. This report makes no identification of Con(iPRA) with Con(PA).

9 Reproduction, provenance, and limits

From the extracted archive root, the primary construction and full checks run with Python 3 and its standard library:

```
python3 machine/build_verify.py
python3 independent/replay.py
python3 comparison/verify_novelty.py machine/machine.tm --start 0
```

Assertions must remain enabled; do not use `python3 -O`. The primary checker reconstructs its parent and the new machine, verifies all symbolic contracts and exact startup, and compares the frozen table bytes. Adding `-export` regenerates its derived source and certificates. The independent replay checks its manifest, recomputes the physical and command certificates, and compares the recorded outputs byte for byte. No optimizer, solver, network connection, or theorem-prover installation is required for these machine checks.

The independent command algebra check has 164 cases with arbitrary opaque underlying stacks and the exact zero-tail alias. An undecided symbolic comparison is an error, not a guessed branch. The mathematical universal word theorem is given in Section 6; the cases provide a separate executable check of its exact contracts.

Directory	Content
`machine/`	Literal dense/native tables, source, exact construction, parent, folds, startup and deterministic certificates.
`independent/`	Separate direct symbolic tape and command verifiers, recorded proofs and replay.
`comparison/`	Every-archive inventory, table/graph census, graph canonicalization and novelty receipt.
`search/`	Bounded local searches, logical source variants and optional grammar-search evidence; no search result is required by the build.
`report/`	This LaTeX source, its included sections, compiled PDF, and reproduction instructions.
`references/`	Attributed inherited calibration/command arguments, selected upstream proof sources, and source licenses.

The source snapshot, archive hashes, exact imported paths, and unchanged license notices are recorded. The 191-token emitter and seven folds are from the original PA195 archive; the controller transformation and its compiler are from the earlier Combined and cross-archive reports; 195S/195C and the independent checker are from the latest 195-state research archive. The present novelty is their verified diagonal-plus-Combined realization at 195 states, its corpus comparison, and the accompanying bounded search. It is not attributed as a new axiom system or the first Combined controller.

The remaining formalization task is to assemble the new literal-table, source-command, and arithmetic-translation steps as one closed Lean or Rocq theorem of the PA biconditional. The existing native 202-to-195 Lean theorem concerns the old table; changing six rows does not automatically instantiate it for the new one. The new finite fold certificate and the independent physical proof support the ordinary mathematical lifting given here. No soundness claim is inferred merely from a long nonhalting prefix, and no state-optimality claim is made.

A Complete transition table

An entry wDq' means write w, move D, and enter q'. The initial state is 0. The symbol H denotes the separate halt label 195. Read down each panel, then continue with the panel on its right. These entries are generated directly from the delivered table bytes.

q	Read0	Read1	q	Read0	Read1	q	Read0	Read1	q	Read0	Read1
0	1R1	1R2	49	0R68	0R69	98	0R111	1R75	147	0R157	1R155
1	1R3	1R1	50	0R70	0L48	99	0L63	0R109	148	0R158	1R159
2	0R0	1R2	51	0R64	1R53	100	0L62	1R113	149	0R31	1R52
3	0R4	0R1	52	0L71	0L15	101	0L90	1R114	150	0R160	1R149
4	1R5	1L6	53	0R72	0L10	102	0R115	0L14	151	0R161	1R152
5	1L7	1L8	54	0R73	1R74	103	0R69	1R103	152	0R162	1R163
6	0L7	1L6	55	0R75	1R76	104	0R116	1R95	153	0L164	1R41
7	0L9	1L6	56	0R77	1R78	105	0R116	1R67	154	0R107	1R85
8	0L10	0L11	57	0R79	1R80	106	0R117	0L14	155	0R165	1R41
9	1R12	0L9	58	0L81	0R1	107	0R118	1L71	156	0R166	1R135
10	0L9	0L10	59	0R82	1R59	108	0R119	1R120	157	0R145	1R167
11	1L8	1L11	60	0L73	1L46	109	0R14	1R109	158	0R168	1R41
12	0R12	0R13	61	0R83	1R84	110	0R66	1R110	159	0R165	1R107
13	0L14	0R13	62	0R54	1R85	111	0R121	1L35	160	0R92	1R111
14	1L15	0L15	63	0L86	1R87	112	0R122	1R123	161	0R134	1R169
15	0L15	1L16	64	0R58	0R82	113	0R88	0L14	162	0R170	1R171
16	0L17	1L16	65	0R88	1R30	114	0R90	1L95	163	0R167	1R165
17	0L18	1L16	66	0R89	1R66	115	0R124	1R124	164	0R135	1R130
18	0L19	1L16	67	0L90	1L33	116	0L63	0R103	165	0L172	1L173
19	0L20	1L16	68	0L81	0R2	117	0R68	0R91	166	0R97	1R174
20	0L21	1L16	69	0R91	1R69	118	0R58	0R91	167	0R145	1L131
21	0L22	1L16	70	0R72	1R92	119	0L86	1R125	168	0R145	1R141
22	0L23	1L16	71	0R46	1R93	120	0R126	1R127	169	0R175	1R173
23	0R23	1R24	72	0R42	0L13	121	0R58	0R110	170	0R88	1R176
24	0R25	1R26	73	0R94	1R37	122	0R99	1R113	171	0R177	1R145
25	0R27	1R28	74	0R30	1R95	123	0R79	1R78	172	0R177	1R130
26	0R29	1R30	75	0R42	1R96	124	0R88	1R88	173	0R178	1R167
27	0R31	1RH	76	0R97	1R98	125	0R128	1R129	174	0R88	1L147
28	0L32	1R33	77	0R99	1L100	126	0R32	1R36	175	0R76	1R179
29	0R34	1R35	78	0R30	1R77	127	0R31	1R111	176	0L180	1R164
30	0L36	1L37	79	0R93	1R101	128	0R130	1R131	177	0R41	1R41
31	0R38	1R39	80	0R30	1R102	129	0R132	1R133	178	0R156	1R166
32	0R40	1R41	81	1R81	1L15	130	0R134	1R135	179	0R181	1R180
33	0R42	1R43	82	0R103	1R82	131	0R136	1R137	180	0R182	1R183
34	0R44	1R45	83	0R94	0L7	132	0R138	1R139	181	0R184	1R185
35	0R46	1R33	84	0R104	1R105	133	0R140	0L10	182	0R186	1R187
36	0R47	1R48	85	0R42	1R106	134	0R97	1R35	183	0R188	1R187
37	0R49	1R50	86	0R107	1R48	135	0R76	1R141	184	0R41	1R189
38	0R51	1R42	87	0R108	0L72	136	0R142	1R143	185	0R176	1R143
39	0R52	1R53	88	0R68	0R66	137	0R143	1R144	186	0R134	1R168
40	0R54	1R55	89	0R109	1R89	138	0R145	1R146	187	0R174	1R190
41	0R56	1R57	90	0R49	1R100	139	0R147	1R148	188	0R145	1R153
42	0R58	0R59	91	0R110	1R91	140	0R149	1R150	189	0R191	1R192
43	0R60	0L14	92	0R30	1R48	141	0R41	1R124	190	0R172	1R88
44	0R61	1R62	93	0R47	1R60	142	0R28	1R151	191	0R32	1R193
45	0R63	1R52	94	0L63	0R89	143	0L139	1L152	192	0L169	1R153
46	0R64	1R65	95	0R94	1L111	144	0R153	1R88	193	0R192	1R194
47	0R58	0R66	96	0R101	0L14	145	0R97	1R154	194	0R188	1R172
48	0R67	0L14	97	0R36	1R112	146	0R155	1R156			

B Archive-by-archive comparison

The following covers all 32 top-level incoming ZIPs at the pinned revision. Names and hashes, nested provenance, and the complete literal-table census are preserved in the machine-readable comparison files.

The filenames below identify the source documents. Exact enclosing-archive hashes, table-member paths, graph identities, and duplicate occurrences are in `comparison/archive_comparison.json` and `comparison/graph_classes.json`. "Report-positive" records a manuscript's supported construction claim; the table does not upgrade every such claim to a newly kernel-checked PA theorem.

Incoming archive	Contribution and outcome
`Below_U208_Research_2026-10-29.zip`	Source-boundary tape windows, head-offset quotient arguments, and guarded source pruning reduce 3,909 source sites to 3,904. The two inherited U208 classes remain at 208 states; no accepted 207-state construction results.
`ConPA_below208_research_2026-10-29.zip`	Compares the original U208 classes, gives a finite-model obstruction to identifying the two cover roles, and checks 279,936 parameter assignments plus local two-row folds. No reduction below 208 is obtained; the negative screens have explicitly restricted templates.
`PA195_ConPA_research_2026-11-14.zip`	Supplies 195S and 195C. Reassociating the unchanged 190-token separate-head word gives 67 readers and six compatible folds. The second table adds H-only Shift and MP output in H. Full source, startup, and independent physical certificates support two distinct 195-state graphs; no new full kernel PA biconditional is claimed.
`PA_consistency_TM_research_2026-11-13.zip`	Gives an original-source 159-node grammar with maximum zero run five, a 157-node combined-negation/MP grammar, and opposite-comparison-residual MP. The source transformations preserve the stated control-flow contracts but do not produce a validated physical table below 196.
`U208_ConPA_2026-10-29.zip`	Gives a three-cell history fold identifying native 211 with 172, with exact startup and symbolic contracts. Its final table is a presentation of original U208 class A, not an independent reduction to compose with the later A copies.
`U208_ConPA_Conditional_Context_2026-10-29.zip`	Develops the conditional-entry argument and supplies both original U208 classes A and B. The same history-dependent state identification accounts for the saving in each; duplicated presentations do not yield additional savings.
`U208_ConPA_Entry_Context_2026-10-29.zip`	Gives a complete entry-context proof and two final tables. Exact graph comparison identifies them with the same original A/B classes. The proof presentation differs, but the physical saving is the same.
`U208_ConPA_History_2026-10-29.zip`	Presents the incoming-history invariant and local positive-block simulation for the original U208 fold, together with its calibration lineage. It supplies no further independent state deletion beyond the same A/B constructions.
`conpa_tm_comparison_and_reduction_search.zip`	Investigates MP-result-in-H and MP-without-zero-push variants, register assignments, and layouts. The MP designs are explicitly experimental in this packet. Two proposed 196-state Shift layouts are rejected more decisively: both halt on blank at transition 43,563.

Continued on the next page

Incoming archive (continued)	Contribution and outcome
conpa_tm_comparison_and_search.zip	Supplies an A/D physical-register permutation at 196 states, a two-rotation 195-state diagnostic, and a logical countermodel. The smaller diagnostic fails 93 of its claimed source contracts; its abstract ideal history router is not a literal successful machine.
conpa_tm_study_2026-11-13.zip	Constructs Combined196 by joining H-only Shift with successful-MP output in H: 3,788 source sites and complete command, startup, and symbolic checks. It is a useful controller improvement at the same state count. Its shorter negation experiment costs 197 states.
pa195_conpa_2026-11-13.zip	Supplies original PA195: the unchanged diagonal 191-token word, 3,700 instructions, 68 readers, and seven valid simultaneous folds. The report provides the ordinary PA-calibration proof and exact runtime/startup certificates. Later repository modules formalize its native quotient.
pa196_research_2026-11-13.zip	Gives an exact 157-node representation of the original source control-flow graph and studies an HA-strength reference calculus. The smaller source DAG loses physical reader-sharing advantages; no sub-196 transition table is validated.
pa208_research_2026-10-29.zip	Adds eighteen U208 helper variants and substantial source/theory research. Together with A/B these yield twenty designated 208-state graph classes. The extra choices are same-size implementations, not accepted 207-state reductions.
pa_below196_research.zip	Updates the earlier census, analyzes eight equivalent separation gates, and proves a restricted Boolean formula-size result while screening grammars and physical allocations. No accepted 195-state construction appears; earlier failed 195-state files are retained as diagnostics.
pa_consistency_comparison_2026_11_13.zip	Splits modal-B and explosion contributions, supplies finite Hilbert proofs, and studies twelve compositions including a 189-token joint gate. The tested physical implementations use 196–200 states. Logical or token-count savings alone do not give a smaller machine.
pa_consistency_comparison_and_new196_2026-11-13.zip	Introduces diagonal-vacuity and separate-head emitters with 191 and 190 tokens, scanner-oriented layouts, and stronger simultaneous-fold checks. It supplies several new 196-state designs. The naive 195-state short scanner fails its unrestricted runtime contracts.
pa_consistency_cross_archive.zip	Combines each diagonal/separate AX emitter with the H-only Shift and MP-in-H controller, then adds source-26 zero-jump variants. All four complete tables have 196 states, with 3,676 or 3,649 source sites. Full startup, symbolic, and command checks do not reduce the count further.
pa_consistency_reduction.zip	Supplies rebracketed196 and H-only196, studies an integrated-witness gate, and gives an internal-converse manuscript. The gate layouts cost 203 or 206 states. Two retained 195-state physical diagnostics fail 714 and 777 source contracts respectively.
pa_consistency_reduction_audit_2026-11-14.zip	Combines diagonal/separate AX with MP-in-H, with and without shorter negation, producing two 196-state and two 197-state tables. It exhausts stated finite-window helper templates and runs restricted writable-scanner synthesis. No accepted 195-state result follows.
pa_consistency_research.zip	Gives 158-node source grammars and a scanner-repaired 196-state layout. The nominal 195-state version confuses a run of seven address zeros with the header marker. Repairing that specific layout requires an extra scanner state.
pa_consistency_research_2026-10-29.zip	Certifies a source specialization from 3,909 to 3,904 primitive sites by inductive register invariants. The resulting source change does not provide a validated physical construction below 208 states.

Continued on the next page

Incoming archive (continued)	Contribution and outcome
pa_consistency_research_2026-11-13(2).zip	Supplies a twelve-rotation 196-state layout, improves one diagonal implementation from 199 to 198, and studies successful-MP halting and joint register abstraction. These results do not improve on the 196-state frontier.
pa_consistency_research_2026-11-13.zip	Develops the diagonal-vacuity calculus, a 3,700-site 199-state implementation, and shorter-negation197. The logical simplification is later used by the 196/195 descendants, but its initial physical allocation is larger.
pa_consistency_research_2026-11-14.zip	Gives NEG18: an 18-instruction encoded-negation routine replacing 47-site blocks at eight occurrences, reducing the separate-head source to 3,441 sites. It supplies a scratch-zero invariant and a complete 197-state implementation: $134 + 68 - 5 = 197$.
pa_tm_comparison_and_reductions.zip	Supplies two different source-CFG-preserving rotated 196-state layouts and a 3,773-site distributed-gate source. The latter compiles to 198 states; the rotated layouts remain at 196.
pa_tm_extended_research_2026-11-13.zip	Constructs four 196-state closing-jump specializations using forced-zero registers CD3, CD4, CD5, or CD6. All four modify the same native row 1365/1. They are alternative implementations; their putative savings cannot be added.
pa_tm_research_2026-11-13(2).zip	Constructs zerojump196 using the source-26 invariant $R_6 = 0$, examines joint jump actions, and builds a 202-state base-eight parity runtime. A wrong-opcode 195-state fold is rejected. The parity runtime lacks a certified sound blank initializer.
pa_tm_research_2026-11-13.zip	Supplies alternative196 with head order $(1, 2, 3, 5, 6, 4)$ and shorter source paths, rebuilds the physical Lean subset, and develops a direct PA proof translation. It remains a same-size construction; the full internal biconditional is not newly exported by the kernel.
pa_unary_power_cover.zip	Proves mutual PA interpretations for a unary weak-power-covering theory and gives uniform proof-construction templates plus an arithmetic emitter. This is a logical/theory contribution; it supplies no new literal machine state saving.
u208_comparison_research_2026-10-29.zip	Compares the original two U208 classes, studies weak/exact covering, and provides a frame counterexample to a rejected 207-state proposal. It predates the eighteen additional helper variants and contains no accepted sub-208 table.
u208_research_2026-10-29.zip	Supplies the original two U208 outputs and 5,084 arbitrary-register symbolic contracts per table. Both use the same incoming-history saving already present in the other original U208 packages.

B.1 What the comparisons rule out, and what they preserve

James Chang's PA209 and the U209 parent of the U208 line are distinct programs. The former has 3,948 source sites; PA196 has 3,812. PA196's reduction uses transposed universal coding, guarded equality vacuity, altered head and antecedent orders, and six complementary reader folds. Those six folds are already included in its published budget of 36 engine states, 13 leaf routers, and 147 reader states. U208's native 172/211 fold has a mixed incoming-direction motif absent from PA196, so it cannot be transferred by merely renaming labels. This excludes that specific transplant, not every history-sensitive quotient.

The six rejected 195-state graph classes must also remain separate from the three prior positive designs. Besides the two-rotation and 714/777-failure cases above, they include the seven-zero scanner, the

separate-head short scanner with 96 failed contracts, and a wrong-opcode zero-jump fold. The last changes native 27/1 to `1 R 62`; source site 26 then increments register 6 and selects site 27 instead of implementing the intended zero decrement to site 34. These witnesses refute the advertised simulations; most do not prove that every conceivable alternative account of the table's blank behavior is impossible.

One scanner audit corrects a particularly important diagnostic error. A tape mismatch at the *first* return to the dispatcher need not refute an instruction implementation: the correct parent can return at 134 and 290 steps with intermediate tapes and reach its required full endpoint only at 574 steps. Positive-block simulation requires the specified complete endpoint, not an arbitrarily selected intermediate return.

The exact corpus census, ordinary mathematical calibration arguments, executable arbitrary-register contracts, and kernel theorems have different roles. The imported formal PA196 endpoint establishes the internal implication from nonhalting to Con(PA). Its converse needs the uniform proof-code and trace constructions described in the supplied manuscripts and in this report. Neither a finite blank run nor a negative bounded search supplies that implication or a state-count lower bound.

C Safe changes to the axiom emitter

This note concerns the 191-token bundled-diagonal emitter in `pa195_conpa_2026-11-13.zip`. Tokens are indexed from zero. The initial operand reads occur at positions 5 (`readH`) and 9 (`readDCons`). The six subsequent head blocks have half-open spans

$$[18, 35), [35, 54), [54, 70), [70, 86), [86, 102), [102, 121).$$

Each contains only `copyA`, `copyD`, `copyH`, `cons`, `push` and `incT`. Consequently each block preserves the parameter registers, pushes the previous top once, and leaves its stated formula as the new top. This is an exact free-algebra identity, independently checked by `source_variants.py`.

Let the six formulas in emission order be $B_1, \ldots, B_6$. For every permutation $\pi \in S_6$, replacing these blocks by $B_{\pi(1)}, \ldots, B_{\pi(6)}$ leaves the final gate unchanged and preserves the number of generated entries. The final AX stack, listed from the top downward, becomes

$$[B_{\pi(6)}, \ldots, B_{\pi(1)}, \rho_8, U(d, h), t] ::_C w.$$

The saved top, generalization and restricted-propositional head retain their positions. Every generated head is either an implication code, which has residue zero modulo four, or the universal code $U(d, h)$, which has residue two. No generated head is one. The unchanged guard and cleanup remain at the end of AX.

Why enumeration completeness survives. The stack words for Push, Pop, Discard and generalization depend only on the number of generated entries and the last two entries, so they are unchanged. An old head B_k formerly had top rank $7 - k$; its new top rank is $7 - \pi^{-1}(k)$. Replace this constant in its axiom selector. All tests during the selector encounter generated formulas different from one; the selected formula and arbitrary underlying stack are preserved. The compiler for modus ponens and theorem generalization is unchanged. Thus there is a primitive-recursive compiler of every old calculus proof into a new command word, verified by the same induction on the finite proof tree. Soundness follows from the proof-store invariant because the set of generated axioms is unchanged. Both arguments are formalizable in PA. Source association and physical allocation still require their separate control-flow and tape-contract checks: a head permutation is not a proof that an arbitrary compiled binary table is correct.

Antecedent exchange. For each of heads 3, 4 and 6, replace $A \to (B \to C)$ by $B \to (A \to C)$. The fixed implicational calculus proves each from the other uniformly, so this replacement yields primitive-recursive proof translations in both directions. The macro-token counts do not change. The generated file `safe_variants.json` contains all combinations of these replacements with the following restricted-propositional parameter exchange.

The restricted-propositional head. The emitted formula is

$$((H \to d) \to D) \to ((D \to H) \to (A \to H)), \qquad d \le D.$$

Exchanging A, H gives

$$((A \to d) \to D) \to ((D \to A) \to (H \to A)).$$

The dominated pair d, D is unchanged. This is the same axiom family under a permutation of its two unrestricted arguments. Its selector chooses the input parameter and H offset in the opposite order after clearing the cache, which still realizes every required tuple.

Dual diagonal seeds. The two formulas

$$\neg U(A, A) \to \neg A, \qquad \neg A \to U(A, \neg A)$$

can each replace $\Gamma(A) = A \to U(A, A)$ in the guarded bundle. The first is ordinary contraposition. For the second, use the retained lemma that vacuity at a fixed variable passes to negation, whose proof uses universal distribution, theorem generalization, and the retained head $(\exists x\, \forall x B) \to B$. Applying it twice and using double negation elimination proves equivalence with $\Gamma(A)$. These seeds are longer (197 total macro tokens), but repeated negation subtrees may have a different sharing cost. They are search candidates, not advertised state reductions.

D Why replacing the equality witness by reflexivity is not immediate

Replacing $\exists x_D E(H, D)$ by $E(H, H)$ does not preserve the existing axiom interface. Here is a countermodel to derivability of the old witness from the remaining heads, gates and reflexivity. Let variable x_1 range over $\{0\}$ and every other variable range over $\mathbb{N}$. Interpret $E(a, b)$ as numerical equality and $M(a, b)$ as bit x_b of x_a. Quantification at label i ranges over its indicated domain. Each domain is nonempty. All the retained propositional, quantifier, congruence and reflexivity heads are valid. Code freshness makes every diagonal vacuity instance valid.

For a gate whose formula code is $c \neq 1$, its witness variable ranges over all natural numbers. Ordinary finite bit comprehension constructs its witness, with the membership index restricted to the domain of its bound variable when necessary. For $c = 1$, the decoded property is the uniformly false formula $M(0, 0)$. Choosing set witness zero satisfies pointed separation. Thus all guarded gates remain valid. Nevertheless

$$\exists x_1\, E(0, 1)$$

is false at an assignment $x_0 = 1$. The old witness schema is therefore not derivable in this proposed replacement calculus. This countermodel does not by itself settle its consistency strength; it rules out the simple reuse of the existing completeness proof.

References

[1] Vladimir Reshetnikov and ProveIt contributors. PA research and incoming archives. Snapshot 5ed5a41fa5932433111eea096311364c6087e8e3. https://github.com/VladimirReshetnikov/ProveIt/tree/5ed5a41fa5932433111eea096311364c6087e8e3/Computability/BusyBeaver/Research/PA. https://github.com/VladimirReshetnikov/ProveIt/tree/5ed5a41fa5932433111eea096311364c6087e8e3/docs/incoming.

[2] James Chang. *Nonhalting and PA-independence for a 196-state Turing machine*. Upstream revision 407cdc356f6b586635e71ac92be394255f4bfea4, imported under jamesyc-upstream/pa196/ in the pinned PA tree. https://github.com/jamesyc/BusyBeaver/tree/407cdc356f6b586635e71ac92be394255f4bfea4.

[3] ProveIt incoming research. pa195_conpa_2026-11-13.zip. The original diagonal PA195 table, seven-fold certificate, and report/sections/pa_calibration.tex. Included in the pinned corpus. https://github.com/VladimirReshetnikov/ProveIt/blob/5ed5a41fa5932433111eea096311364c6087e8e3/docs/incoming/pa195_conpa_2026-11-13.zip.

[4] ProveIt incoming research. pa_consistency_comparison_and_new196_2026-11-13.zip. The diagonal-vacuity and separate-head numerical calculi and their compilers; included in the pinned corpus.

[5] ProveIt incoming research. conpa_tm_study_2026-11-13.zip. Especially chapters/combined_source_macros.tex: H-only Shift, MP-result-in-H, and arbitrary-value loop proofs.

[6] ProveIt incoming research. pa_consistency_cross_archive.zip. Especially independent_review/command_compiler_note.tex: the uniform compiler for nine- and ten-entry AX stacks.

[7] ProveIt incoming research. PA195_ConPA_research_2026-11-14.zip. 195S and 195C, independent symbolic tape checker, and independent_verification/composite_command_audit.tex. The archive's UTC date differs from the report's Pacific-date convention.

[8] ProveIt. pa195-global/README.md and pa195/ipra/README.md. Scope of the existing native quotient and arithmetic coding developments. https://github.com/VladimirReshetnikov/ProveIt/blob/5ed5a41fa5932433111eea096311364c6087e8e3/Computability/BusyBeaver/Research/PA/pa195-global/README.md.

[9] ProveIt. pa196-vs-u208/PA-BRIDGE.md and the neighboring reverse proof-store and finite-stuttering audits, in the pinned PA tree.

[10] ProveIt. ZFC small-machine research and inherited register-engine lineage. https://github.com/VladimirReshetnikov/ProveIt/tree/5ed5a41fa5932433111eea096311364c6087e8e3/Computability/BusyBeaver/Research/ZFC.

[11] ProveIt. *ConciseIPRA*. https://github.com/VladimirReshetnikov/ProveIt/tree/5ed5a41fa5932433111eea096311364c6087e8e3/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.